title: Lattice Kinetic Monte Carlo Assignment short_title: Lattice KMC Assignment authors: gvarnavides date: 2026-06-02

In this assignment, we will investigate how interacting particles organize on a lattice using probabilistic simulations. The goal is to connect ideas from statistical physics, such as energy, temperature, entropy, and chemical potential, to the microscopic dynamics of a many-particle system.

We consider a simple model of a binary mixture on a square lattice. Each lattice site may contain either a blue particle, a red particle, or a vacancy (an empty site). The particles interact with their neighbors, and thermal fluctuations allow the system to evolve over time.

Lattice Kinetic Monte Carlo

Kinetic Monte Carlo (KMC) is a stochastic simulation method used to model the time evolution of systems with thermally activated dynamics. Rather than solving equations of motion directly, KMC evolves the system through a sequence of probabilistic microscopic changes, such as particles moving or changing state.

In our lattice model, each site i is assigned a variable $s_i \in -1, 0, 1$, where $s_i = -1$ corresponds to a blue particle, $s_i = 1$ corresponds to a red particle, and $s_i = 0$ corresponds to a vacancy.

The system evolves through local updates, such as exchanging a particle with a neighboring vacancy. Whether a proposed move is accepted depends on how it changes the total energy of the system.

At low temperature, the system tends to evolve toward ordered low-energy configurations. At higher temperature, thermal fluctuations become stronger and disorder is favored. KMC therefore allows us to study how microscopic interactions compete with thermal entropy.

Pairwise Hamiltonian

To describe the energetics of the lattice, we define the total energy function of the system

$$H = \sum_{\langle i,j \rangle} J_1 s_i s_j + \sum_{\langle\langle i,j \rangle\rangle} J_2 s_i s_j - \sum_i (\mu_B \delta_{s_i,-1} + \mu_R \delta_{s_i,1}).$$

This expression contains several terms:

- The first sum runs over all nearest-neighbor pairs $\langle i, j \rangle$.
- The second sum runs over next-nearest-neighbor pairs $\langle\langle i, j \rangle\rangle$.
- J_1 and J_2 determine the interaction strength between neighboring sites.
- The quantities μ_B and μ_R are chemical potentials that control how favorable it is to add blue or red particles to the system.
- $\delta_{a,b}$ is the Kronecker delta, equal to 1 when $a = b$ and 0 otherwise.

The products $s_i s_j$ determine whether neighboring sites contribute positively or negatively to the energy. Depending on the signs of J_1 and J_2, the system may favor clustering, checkerboard ordering, phase separation, or more complex structures.

In the first part of the assignment, we will work in the canonical ensemble, where the total number of particles is fixed. Later, we will introduce grand-canonical simulations, where the chemical potentials μ_B and μ_R allow particles to enter or leave the system.

Metropolis Algorithm

The lattice is evolved using the Metropolis-Hastings algorithm. For each proposed update, we compute the corresponding energy change: $\Delta E = E_{\text{new}} - E_{\text{old}}$.

The move is then accepted with probability:

$$P_{\text{accept}} = \begin{cases} 1, & \Delta E \le 0, \\ \exp(-\Delta E / k_B T), & \Delta E > 0. \end{cases}$$

This means:

- Moves that lower the energy are always accepted.
- Moves that increase the energy may still occur because of thermal fluctuations.
- Higher temperatures make energetically unfavorable moves more likely.

This update rule ensures that the simulation samples configurations according to the Boltzmann distribution:

$$P(H) \propto \exp(-H / k_B T).$$

Over many Monte Carlo steps, the system evolves toward thermal equilibrium while still exhibiting fluctuations and local dynamics.

```
# Imports
from lattice_kmc import initialize_lattice, run_simulation, plot_lattice_disks, interactive_kmc_canvas, compute_
```

```
import matplotlib.pyplot as plt
import numpy as np

MODE_CANONICAL = 0
MODE_GRAND_CANONICAL = 1
MODE_SURFACE_GRAND_CANONICAL = 2
```

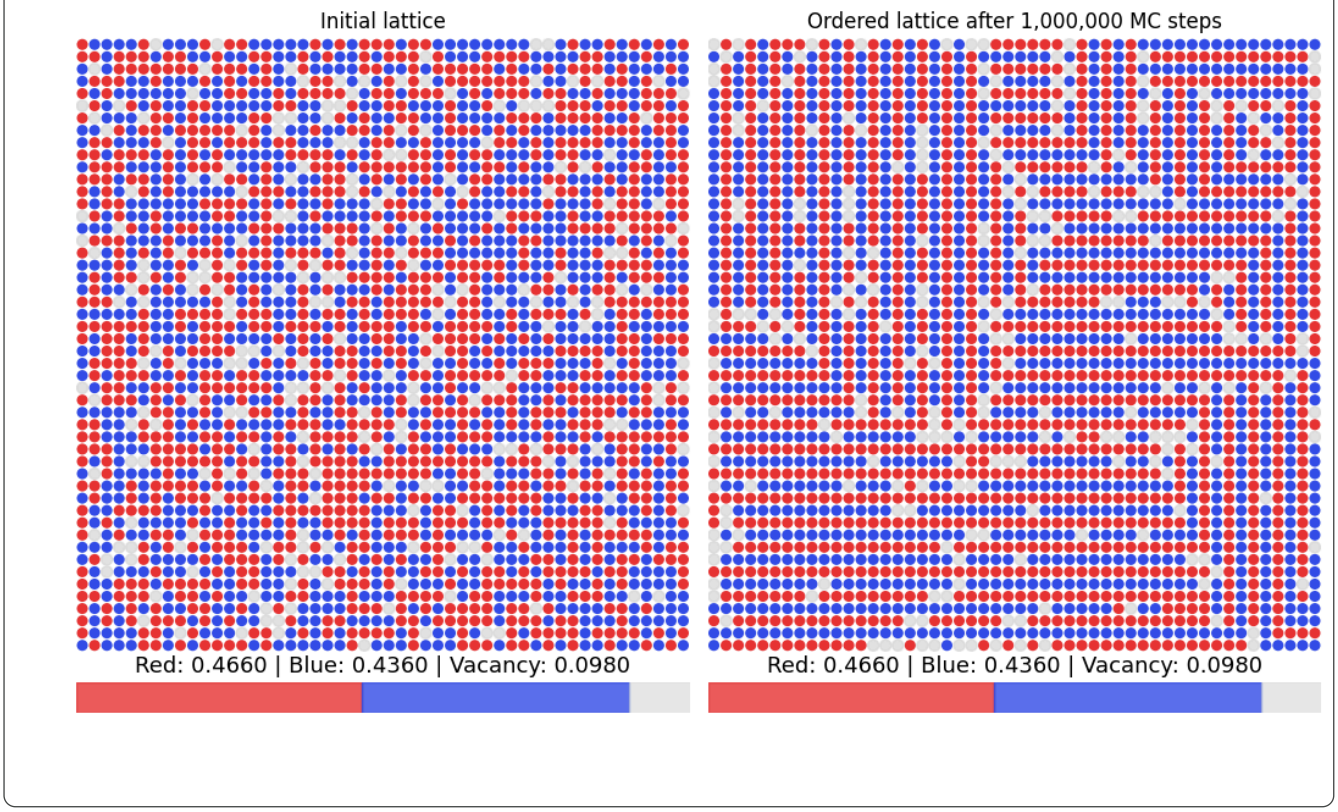
Example Simulation

We begin with a square lattice initialized with randomly distributed blue particles, red particles, and vacancies according to the probabilities $(0.45, 0.45, 0.1)$

```
#| label: square_lattice_example
# Initialize square lattice
nx, ny = (56,56)
lattice,vacancy_coords = initialize_lattice((nx,ny),seed=2026)

# 1,000,000 steps at constant composition and temperature
ordered_lattice, ordered_vac = run_simulation(
    lattice,
    vacancy_coords,
    n_steps = 1_000_000,
    T=5.0,
    J1=-10.0,
    J2=10.0,
    mode= MODE_CANONICAL
)

fig, axs = plt.subplots(1,2,figsize=(10.5,6))
plot_lattice_disks(lattice,ax=axs[0])
plot_lattice_disks(ordered_lattice,ax=axs[1])
axs[0].set_title("Initial lattice")
axs[1].set_title("Ordered lattice after 1,000,000 MC steps")
fig.tight_layout()
```



Part 2a: Canonical Ensemble Ordering Phases

In this exercise, we will investigate how different interaction parameters produce different types of lattice ordering. Run lattice KMC simulations at fixed composition and a normalized temperature $T = 5.0$ to obtain equilibrated lattices for the following interaction parameters:

- $J_1 = -10,\ J_2 = 0$
- $J_1 = -10,\ J_2 = 10$
- $J_1 = 10,\ J_2 = -10$

For each simulation:

1. Plot the final lattice configuration.
2. Compute and plot the structure factor

$$S(\mathbf{q}) = |F_{\mathbf{r}\to\mathbf{q}}\,[s(\mathbf{r})]|^2,$$

   where F denotes the discrete Fourier transform.
3. Compute and plot the horizontal and vertical correlation functions

$$C_h(r) = \langle s(i,j)\, s(i+r,j)\rangle\,,$$

   where the average is taken over all lattice positions.
4. Compute the following scalar order parameters:
   - Checkerboard ordering:

$$m_{\mathrm{AF}} = \frac{1}{N}\sum_{i,j}(-1)^{i+j}s(i,j)$$

   - Vertical stripe ordering:

$$m_{\mathrm{stripe},v} = \frac{1}{N}\sum_{i,j}(-1)^{i}s(i,j)$$

   - Horizontal stripe ordering:

$$m_{\mathrm{stripe},h} = \frac{1}{N}\sum_{i,j}(-1)^{j}s(i,j)$$

Here, N is the total number of lattice sites.

Discuss how the real-space ordering patterns relate to the peaks observed in the structure factor and the values of the order parameters.

Part 2b: Grand-Canonical Ensemble

In the grand-canonical ensemble, the total number of particles is no longer fixed.
Instead, the chemical potentials μ_B and μ_R control how favorable it is for blue and red particles to occupy lattice sites. In this exercise, we will investigate how the lattice composition changes as the chemical potential is varied.

Run lattice KMC simulations using: $J_1 = -2.5,\ J_2 = 0,\ T = 2.5$ in `MODE_GRAND_CANONICAL`, while varying $\mu_B = -\mu_R \in [-1.5,\ 1.5]$.

For each value of μ_B:

1. Run the simulation until the system reaches equilibrium.
2. Compute the fraction of blue sites

$$f_B = \langle \delta_{s,-1} \rangle\,,$$

   where $\delta_{s,-1} = 1$ for blue sites and 0 otherwise.

   You may use the `compute_fractions` function for this calculation.
3. Repeat the simulation multiple times and average the results to reduce statistical noise.
4. Plot the average blue-site fraction f_B as a function of μ_B.
5. Numerically estimate and plot the susceptibility-like response

$$\frac{\partial f_B}{\partial \mu_B}$$

   as a function of μ_B.

Discuss the following questions:

- Why does increasing μ_B favor blue particles?
- Why is the curve symmetric about $\mu_B = 0$?
- What does the peak in

$$\frac{\partial f_B}{\partial \mu_B}$$

  indicate about the sensitivity of the system to changes in chemical potential?

Part 2c: Heat Capacity

In this exercise, we will study how thermal fluctuations change with temperature by computing the heat capacity of the lattice system. Run lattice KMC simulations using: $J_1 = -2.5,\ J_2 = 0,\ \mu_B = -\mu_R = 0.5$ in `MODE_GRAND_CANONICAL`, while varying the temperature $T \in [1,\ 10]$.

For each temperature:

1. Run the simulation until the system reaches equilibrium.
2. Use the `total_energy` function to compute the total lattice energy E during the simulation.
3. Estimate the average energy $\langle E \rangle$ and the average squared energy $\langle E^2 \rangle$.
4. Repeat the simulation multiple times and average the results to reduce statistical noise.
5. Compute the heat capacity

$$C(T) = \frac{\langle E^2 \rangle - \langle E \rangle^2}{T^2}.$$

6. Plot $C(T)$ as a function of temperature.

Discuss the following questions:

- Why does the heat capacity measure energy fluctuations?
- At which temperatures are the fluctuations largest?
- How does the lattice ordering change as the temperature increases?
- Does the peak in $C(T)$ suggest the presence of a phase transition or crossover?

Part 2d: Surface Grand-Canonical Ensemble

In this exercise, we will explore non-equilibrium effects in a surface-driven grand-canonical simulation.

Use the interactive widget below to run the following KMC simulations:

- Prepare an initial phase-separated structure by running a canonical ensemble with $J_1 = -10,\ J_2 = 0,\ T = 5.0.$
- Pause the simulation after it has sufficiently phase-separated.
- Continue the simulation by switching to a surface grand-canonical and relatively large chemical potentials $\mu_B = -\mu_R \approx 5.$

Experiment with different temperatures and chemical potentials, and observe how the lattice evolves over time. In particular, look for:

- The formation of interior domains or “islands”
- Competition between different ordered regions
- Slow evolution toward equilibrium
- Metastable states in which the lattice becomes kinetically trapped

Discuss the following questions:

- Why can isolated islands persist for long times?
- Why does the system sometimes become trapped in partially ordered states?
- How does temperature affect the ability of the lattice to escape these metastable configurations?
- What is the difference between thermodynamic equilibrium and kinetic accessibility?

```
#| label: interactive-kmc-widget
interactive_kmc_canvas(
    lattice,
    vacancy_coords,
    run_simulation,
    T=5.0,
    J1=0.0, J2=0.0,
    muB=0.0, muR=0.0,
    mode=MODE_CANONICAL,
    steps_per_frame=100,
    interval=100,
)
```

```
VBox(children=(HBox(children=(Play(value=0, layout=Layout(width='320px'), max=9999, repeat=True), Button(descr…
```